

Chapter 6 Software Design Pattern

What is a Design Pattern?

- is a reusable solution [可复用解决方案](#) to a common problem that software developers often face while designing software.
- It's like a **template** or a **guide** that helps solve problems in a smart, proven way [有效](#)
- like a **blueprint** that can be adapted to fit different situations [可调整的蓝图](#)
- Unlike a ready-made function or library, a design pattern is not direct code but a **reusable idea or concept** that can help to **save time and improve code quality**.

Example: Imagine you are building many houses. Instead of drawing the blueprint from scratch every time, you reuse a standard design for the kitchen and bathroom. Similarly, design patterns give you **standard "blueprints"** to write your program better.

*Design patterns are often confused with algorithms. The **key difference** is that:*

- An **algorithm** is like a recipe with clear steps to follow. [有清晰步骤的菜谱](#)
- A **design pattern** is more like a flexible blueprint that guides you to build your own solution. [一份灵活的蓝图, 指导您构建自己的解决方案](#)

What a design pattern consists of:

1. **Intent** 意图 – A short explanation of the problem and how the pattern solves it.
2. **Motivation** 动机 – A more detailed explanation of the problem and why this solution works [对问题及其解决方案有效性的更详细解释](#)
3. **Structure** 结构 – A diagram or description showing the parts (classes/objects) and how they connect [展示各个部分\(class/obj\)及其连接方式的图表/描述](#)
4. **Code example** – A sample code to help understand how the pattern works in practice.
5. **Other details (optional)** – When to use it, how to implement it, and how it relates to other patterns.

Design Pattern Elements

1. **Pattern Name** 模式名称
A short and meaningful name that represents the pattern (e.g., *Singleton*, *Strategy*).
 - Helps developers talk about the pattern easily and clearly [轻松清晰地讨论模式](#)
 - Improves communication and understanding during software design discussions [促进沟通&理解](#)
2. **Problem (Solved and Applicability)** 适用性
Describes **what kind of design problem** the pattern solves [该模式解决的设计问题类型](#)

- Explains **when** the pattern should be used and **why**. 解释何时用 & 原因
- Describes the **context** of the problem—real-world situations or software issues where the pattern is useful. 描述问题的背景 —现实世界的情况/该模式适用的软件问题
- May include **specific design problems**, common object relationships, or examples. 可能包括具体的设计问题、常见的obj关系/示例
- Can also list **preconditions** 先决条件(requirements) that must be true before applying the pattern makes sense. 还可以列出应用该模式之前必须满足的

3. Solution

Describes **how the pattern solves the problem**. 描述模式如何解决问题。

- It includes the **participants** (classes or objects), their **responsibilities**, and how they **collaborate**. 它包括参与者(class/obj)、它们的职责 & 它们如何协作
- Defines the **structure** and **relationships** in a design, but **does not provide specific code**. 定义设计中的结构&关系, 但不提供具体的代码
- It is an **abstract description** 抽象的描述, focusing on the idea or model, not on the exact implementation. 侧重于想法/模型, 而不是具体的实现

4. Consequences (Effects) 后果(效果)

Describes the **results** of using the pattern—both positive and negative.

- Helps in **evaluating design choices** by showing the **benefits and trade-offs**. 通过展示其优缺点, 有助于评估设计选择
- Explains the **costs** of applying the pattern 应用该模式的成本, such as increased complexity, memory use, or performance impact.
- Important for understanding how the pattern affects system qualities like:
 - **Flexibility** 灵活性 – how easily it can be changed.
 - **Extensibility** 可扩展性 – how easily it can grow with new features.
 - **Portability** 可扩展性 – how easily it can be moved to different environments.

Element	Description
Pattern Name	A clear, short name to identify the pattern and help communication.
Problem	What issue the pattern solves, when to use it, and real-world context.
Solution	Abstract explanation of how the pattern works—roles, relationships, and structure (not code).
Consequences	Effects of applying the pattern, including benefits, drawbacks, and system impacts.

This structure ensures that design patterns are easy to share, understand, and apply across many different software development situations. 这种结构确保设计模式易于在许多不同的软件开发情况下共享、理解&应用

When to Use Patterns?

1. You face a common design problem 面临常见的设计问题

- The **same kind of problem** appears across many projects or systems. 同一类型的问题出现在许多项目/系统中
- Design patterns offer **proven solutions** 有效的解决方案 for these recurring challenges反复出现的挑战, often with some flexibility for variations. 具有灵活性

2. You want to reuse tried-and-tested solutions 复用久经考验的解决方案

- Instead of reinventing the wheel, you apply a solution that's already been shown to work well in similar situations. 与其重复造轮子, 不如采用在类似情况下已被证明行之有效的解决方案
- This **saves time** and **reduces risk of errors**. 节省时间&降低错误风险

3. You aim to improve team communication 目标是改善团队沟通

- Using **standard names** 标准名称 like Observer, Decorator, or Factory makes it easier to discuss architecture and code with others. 可以更轻松地与他人讨论架构&代码
- It builds a **shared understanding** 建立共识 among developers, especially in larger or long-term projects. 尤其是在大型/长期项目中

4. You want more maintainable and flexible code 更易于维护且更灵活

- Patterns help create **modular, scalable, and readable** designs. 模式有助于创建模块化、可扩展且可读性强的设计
- They encourage **loose coupling, better organization, and easier future updates**. 促进松散耦合、更佳的组织结构&更便捷的未来更新

5. You are solving a multi-step or structured problem 解决一个多步骤/结构化问题

- Some problems require more than a simple linear solution 不仅仅需要简单的线性解决方案. Patterns guide how to **structure these steps clearly**. 模式可以指导如何清晰地构建这些步骤
- However, be cautious: if your problem is very simple, a pattern might be unnecessary or too complex (overkill). 如果您的问题非常简单, 模式可能没有必要/过于复杂(过度)

6. You're looking for a direction, not a full solution 正在寻找方向, 而不是完整的解决方案

- Patterns are useful when you care more about **how to approach** a problem than about **detailed implementation**. 更关心如何解决问题而不是详细的实现时
- They act as a **design guide** rather than a complete derivation. 当设计指南, 不是完整的推导

7. You need a temporary or partial solution 需要一个临时/部分解决方案

- In fast-paced or evolving projects, patterns can serve as **temporary bridges** 临时桥梁, helping you keep the design clean while you refine or scale up later. 帮助您在以后改进/扩展时保持设计的简洁

Example: Mediator Pattern

If many classes in your project need to communicate, but you want to avoid them being directly dependent on each other 想避免它们直接相互依赖, you can use the **Mediator Pattern**. It centralizes communication 集中通信 making the system **more modular and easier to maintain**. 更加模块化, 更易于维护

When Not to Use Patterns

- If the problem **only exists in one place** and is **unlikely to recur**, a pattern may be unnecessary. 如果问题仅存在于一处且不太可能再次发生
- If the solution is **simple and straightforward**, patterns might introduce unwanted complexity. 如果解决方案简单明了, 模式可能会带来不必要的复杂性
- If you need a **full, detailed algorithm**, not just a concept or design model, a pattern might be too abstract to help. 如果您需要完整、详细的算法, 而不仅仅是一个概念/设计模型, 那么模式可能过于抽象而无法提供帮助

Design patterns are powerful tools, but like any tool, they are most effective when used in the **right context** 在正确的环境中使用时效果最佳. Use them to solve **recurring, structured problems** with a need for **clear communication and maintainable design** 使用它们来解决反复出现的结构化问题, &需要清晰的沟通&可维护的设计. Avoid using them where a **simple solution will do**. 避免在简单的解决方案即可解决问题的情况下使用它们

Benefits of Design Patterns

✓ Reuse Best Practices

- Design patterns are a **toolkit of proven solutions** 解决方案工具包 to common problems in software design.
- You don't have to invent a solution from scratch. 无需从头开始
- Even if you haven't studied them, you may already be using some patterns without knowing it. 即使从未学习过,也可能已经在不知不觉中使用了
- Learning them lets you apply **expert knowledge** 运用专业知识 more intentionally and effectively.

✓ Improve Communication

- Patterns create a **shared language** among developers. 创建了共享语言
- Saying "Use a Strategy" or "Apply Singleton here" makes your ideas clear instantly.
- It reduces the need for lengthy explanations. 减少了冗长解释
- This is especially helpful in **team environments** and **large projects**. 在团队环境&大型项目中尤其有用

✓ Make Code More Flexible and Maintainable

- Patterns promote **separation of concerns, modularity, and loose coupling**. 促进关注点分离、模块化&松散耦合
- This makes your code easier to modify, test, and extend. 更易于修改、测试&扩展
- You can change one part of the system without affecting others. 改一部分,不会影响其他部分

✓ Speed Up Development

- Solving problems with patterns is often **faster and more reliable** than designing from scratch. 更快速、更可靠
- You avoid common design pitfalls 避免常见的设计陷阱 by relying on **well-documented, battle-tested structures**. 依靠记录详尽、久经考验的结构
- It helps especially with **scaling systems** or **refactoring old code**. 有助于扩展系统/重构旧代码

✓ Support Object-Oriented Design Principles

- Studying patterns deepens your understanding of **object-oriented programming (OOP)**. 学习模式可以加深对 OOP 的理解
- You learn **how and why to apply principles** like inheritance, encapsulation, and polymorphism through real-world examples. 过实际示例学习如何&为何应用继承、封装&多态等原则
- This improves your overall design thinking, even beyond the patterns themselves. 无需提升您的整体设计思维,甚至超越模式本身

Example: Strategy Pattern

If you're creating multiple sorting algorithms 多种排序算法 (e.g., Bubble Sort, Quick Sort, Merge Sort), the **Strategy Pattern** lets you switch 切换 between these algorithms **without changing the core logic** 无需更改程序的核心逻辑 of your program. Each sorting method becomes a separate class, and the main program can use any of them based on the situation.

Final Thought:

You *can* write code without knowing design patterns 无需了解设计模式即可编写代码—but learning them makes you a **smarter, faster, and more collaborative developer** 学习设计模式可以让您成为更聪明、更快速、更具协作性的开发者 They give you a **deeper toolkit** 更深入的工具包, improve the **quality of your design** 提高设计质量, and make your work more **professional and scalable**. 使工作更具专业性和可扩展性

Drawbacks to Design Patterns

✗ **Not Always Necessary** 并非总是必要的

- Using patterns for simple problems can **overcomplicate** your code. 过于复杂化
- Sometimes a **basic solution** is clearer and more efficient. 基本的解决方案更清晰、更高效
- In such cases, a direct solution would be **faster to write, easier to understand, and more efficient**. 编写速度更快、更容易理解、更高效

✗ **Overengineering Risk** 过度工程风险

- Beginners often try to **force patterns everywhere** 强制使用, even when not needed.
- This can lead to designs that are too abstract or filled with unnecessary parts. 过于抽象或充斥着不必要的部分
- Instead of solving problems, it creates new ones by making the code harder to manage 不能解决问题, 反而会使代码更难管理, 从而产生新的问题

✗ **Can Make Code Harder to Read**

- Patterns can **increase complexity** 增加复杂性, especially when used without a clear need. , 尤其是在没有明确需求的情况下使用
- Someone unfamiliar with the pattern may **struggle to understand** what the code is doing. 难以理解
- This hurts code readability and slows down collaboration or maintenance. 减慢协作或维护的速度

✕ Extra Layers and Objects 额外的层和obj

- Some patterns (like *Facade*, *Decorator*) introduce **extra classes or layers** that may not be justified in small or simple projects. 会引入额外的class或层, 这在小型或简单的项目中可能不合理
- Many patterns add extra classes, interfaces, or structures to the codebase. 添加额外的class、接口或结构
- While this may **help in large systems**, it can clutter smaller applications. 这可能对大型系统有所帮助, 但会使小型应用程序变得混乱
- These added layers may **slow down development** and **reduce performance unnecessarily**. 这些新增的层可能会减慢开发速度, 并降低不必要的性能

✕ Sometimes Redundant in Modern Languages

- Some patterns solve problems that modern programming languages already handle with built-in features. 有些解决到-现代编程语言已经通过内置功能处理的问题
- For example, the Strategy Pattern can often be replaced with lambda functions or simple callbacks.
- Using the full pattern in these cases is like using a complex tool for a simple job. 完整地使用模式就像使用复杂的工具来完成简单的工作

Example:

If your app only has one button and one screen, applying the **Facade Pattern** to "hide complexity" might be unnecessary—it just adds **extra code for no real benefit**. 增加额外的代码, 而没有任何实际好处

Types of Design Patterns?

- can vary based on how complex they are, how much detail they have, and how much of the system they affect. 会根据其复杂程度、细节程度以及对系统的影响程度而有所不同

1. Creational Patterns 创建型 – How to create objects 如何创建obj

- help you control how objects are created, making your code more flexible and reusable. 帮助您控制obj的创建方式, 从而使代码更加灵活和可复用
- Examples:
 - **Singleton** – Make sure only one object of a class exists.
 - **Builder** – Step-by-step creation of complex objects.
 - **Prototype** – Clone existing objects instead of creating new ones.
 - **Abstract Factory** – Create related objects without specifying exact classes.

2. **Structural Patterns** 结构型 – How to organize objects [如何组织obj](#)

- show how to combine objects and classes into larger structures, while keeping them easy to manage. [展示如何将obj和class组合成更大的结构, 同时保持其易于管理](#)
- Examples:
 - **Adapter** – Make two incompatible classes work together.
 - **Decorator** – Add features to an object without changing its code.
 - **Facade** – Provide a simple interface to a complex system.
 - **Composite, Bridge, Proxy, Flyweight** – Help in building scalable and efficient object structures.

3. **Behavioral Patterns** 行为模式 – How objects communicate [obj如何通信](#)

- These focus on how objects interact and share responsibilities. [关注如何交互&共享职责](#)
- Examples:
 - **Observer** – Notify many objects when one changes.
 - **Strategy** – Switch between different algorithms easily.
 - **Command, Iterator, Mediator, State, Visitor, Chain of Responsibility** – Handle communication and control flow in clean, reusable ways.

Singleton Pattern (Creational Pattern)

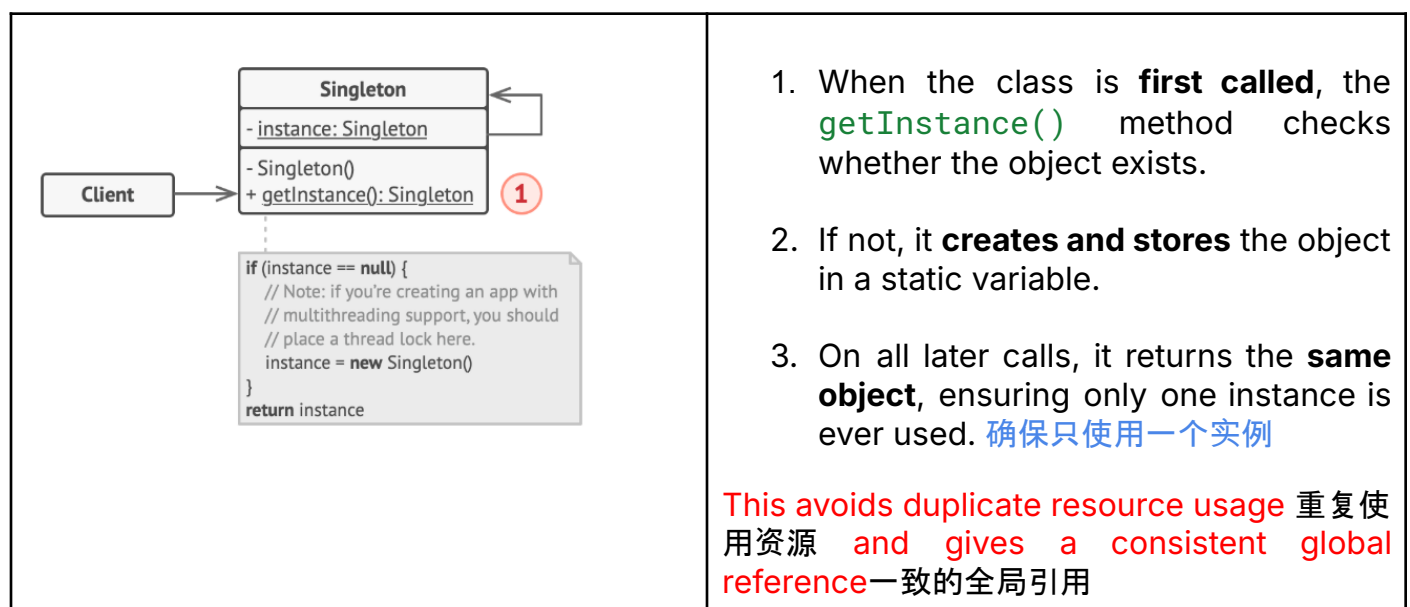
- Ensures only **one object of a class** exists and provides a **global access point** 全局访问点 to it.
- It's useful when exactly one object is needed to coordinate actions across a system. 一个obj来协调整个系统的操作
- It's like a single manager object that everyone in the system shares. 一个系统中所有成员共享的管理器obj

Problem Solved & Applicability

- **Prevents multiple instances** of a class that could cause errors or inconsistencies. 防止class的多个实例可能导致错误或不一致
- In some systems, like a database or logger记录器, creating multiple objects could lead to **conflicting operations or wasted resources**. 可能会导致操作冲突或资源浪费
- Singleton is applicable when **centralized control or shared access** is required across the application. 当应用程序需要集中控制或共享访问

Solution (How It Works)

- Make the class **constructor private** so no one can create new objects directly. 没有人可以直接创建新obj
- Store the **single instance in a static field**. 不一致
- Provide a **public static method** (`getInstance()`) to return that instance 实例 every time.
- This guarantees that only **one shared object** is ever used. 只有一个共享obj被使用



How to Apply

- Identify where only **one instance** is needed (e.g., logging, app settings).
确定只需要一个实例的情况 (日志记录、应用设置)
- Create a private constructor to **block direct creation**.
阻止直接创建实例
- Add a static field and a static method to **manage access** to the instance.
管理对实例的访问

When to Use

- You need **exactly one shared instance** across your system 系统只需要一个共享实例, such as a **printer spooler, logger, or configuration manager**.
打印机后台处理程序、记录器或配置管理器
- You want to **avoid memory waste** by preventing multiple copies.
避免内存浪费 多次复制
- You need **easy access** to a shared resource from anywhere in the program.
从程序中的任何位置轻松访问共享资源
- For example, in a logging system 日志系统, a singleton avoids creating multiple log writers that could duplicate or break log entries. 避免创建可能重复/破坏日志条目的多个日志写入器

Example: In a **printer application**, the print queue should only be managed by **one object**.
打印队列应该仅由一个obj管理

If two managers tried to control the queue at the same time, jobs could be lost or overlap. 试图同时控制队列, 则作业可能会丢失或重叠

The Singleton ensures **only one print manager** handles all print tasks. 确保只有一个打印管理器处理所有打印任务

✓ Controlled Access to a Single Instance 单例访问控制 - Guarantees that only one object controls the resource. 控制资源 - Prevents bugs from having multiple conflicting instances. 防止多个冲突实例导致的错误 ✓ Global Access 全局访问 - The singleton instance is easily accessible from anywhere. 可从任何位置轻松访问 - No need to pass the object through multiple parts of the program. 无需将obj传递到程序的多个部分	✗ Hard to Test 难以测试 - Singletons introduce global state, making unit testing difficult. 引入全局状态, 使单元测试变得困难 - Tests might interfere with each other because they share the same object. 测试共享同一个obj, 测试可能会相互干扰 ✗ Can Hide Dependencies 隐藏依赖关系 - If many classes use the Singleton, it becomes tightly coupled to multiple parts of the system. - This makes your code harder to maintain or refactor. 更难维护或重构
---	--

✓ **Saves System Resources** 节省系统资源

- Since the object is created once, it avoids wasting memory or processing time. 只需创建一次, 因此避免了内存或处理时间的浪费
- Especially helpful for heavy objects like configuration loaders or database connectors. 对于配置加载器或数据库连接器等重量级obj尤其有用

✗ **Risk of Overuse** 过度使用的风险

- Developers may try to use Singleton even when it's not necessary. 不必要的情况
- Overusing it can lead to overengineering or poor design patterns across your system. 整个系统的过度工程或糟糕的设计模式

Mediator Pattern 中介者模式 (Behavioral Pattern)

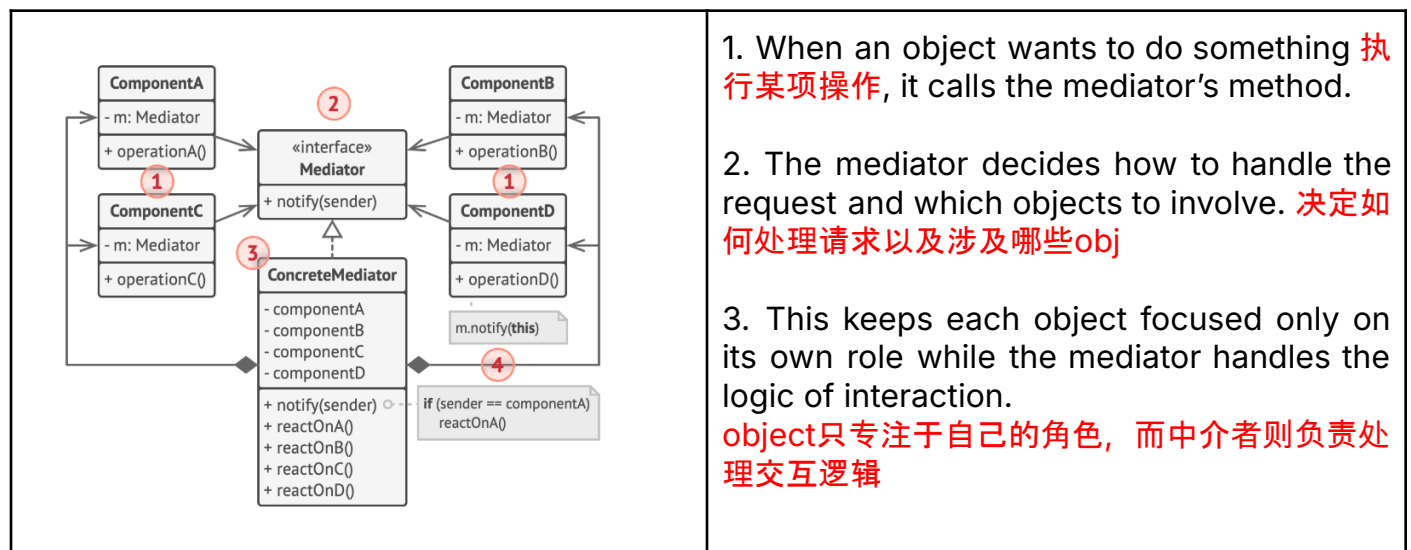
- Provides a central object (mediator) that manages communication between other objects. 来管理其他obj之间的通信
- It helps **avoid direct connections** 避免class直接连接 between classes, making the system easier to change and maintain. 更易于更改和维护
- It's like a **central controller** 中央控制器 that handles messages instead of letting components talk to each other directly. 负责处理消息, 而不是让组件之间直接通信

Problem Solved & Applicability

- **Reduces direct dependencies** between multiple classes that need to interact. 减少需要交互的多个class之间的直接依赖关系
- Without a mediator, objects become **tightly linked**, and changing one may affect others. 变得紧密相连, 更改一个obj可能会影响其他obj
- This pattern is useful when **many objects** need to **coordinate**, but **direct communication** makes the system complex. 多个obj需要协调, 但直接通信会使系统变得复杂时

Solution (How It Works)

- Introduce a middle-man class, called the **mediator**, that acts as a central coordinator for communication between objects. 作为obj间通信的中央协调者
- Instead of talking to each other directly, **all communication between classes goes through the mediator**. 通信都通过中介者进行
- Each class involved in the communication (called colleagues) only knows the mediator and does not need to know about the other classes. 只知道中介者, 无需了解其他class
- The **mediator knows all the participating classes** 了解所有参与的, their roles, and how they should interact, which allows it to control the flow of information 这使得它能够控制信息流
- This setup helps to reduce tight coupling between classes 设置有助于减少class之间的紧密耦合 and keeps the interaction logic centralized and easier to manage. 保持集中化, 更易于管理



How to Apply

- Identify the objects (colleagues) that need to communicate **frequently** 确定需要频繁通信的obj
- Create a mediator class that **handles the interaction logic** between these objects. 处理这些obj之间的交互逻辑
- Make sure each object uses the mediator instead of talking directly to others.

When to Use

- You have many classes that depend on each other and communicate often. 相互依赖且经常通信的
- You want to avoid creating too many direct connections between objects. 避免创建过多的直接连接
- You need a clear and central way to manage interactions 清晰且集中的方式来管理交互, such as in a **chatroom system or air traffic control system**. 聊天室系统/空中交通管制系统

Example: In a chatroom system, users don't send messages to each other directly. 用户之间不会直接发送消息

- Each user sends their message to a mediator, which forwards it to the right user. 会将消息转发给正确的用户
- This keeps the communication clean and avoids each user having to know about all others. 避免每个用户都需要了解所有其他用户的信息

✓ Clear Communication Structure 清晰的通信结构 - Helps organize how objects communicate. 组织通信方式 - Prevents complex webs of direct links between objects. 复杂的直接链接网络 ✓ Easier Maintenance 更易于维护 - Changes in one class don't affect others directly. 更改不会直接影响其他class - Only the mediator needs to be updated to support new logic. 更新中介器=支持新的逻辑 ✓ Improved Flexibility 更高的灵活性 - New objects can be added with minimal changes. 少量更改即可添加新obj - Reduces dependency issues across the system. 减少依赖问题	✗ Mediator Can Become Too Complex - If the logic grows, the mediator may become hard to manage. 难以管理 - All communication passes through it, which can create a bottleneck. ✗ Can Hide Dependencies 隐藏依赖关系 - Because the mediator controls everything 控制一切, it's harder to trace which objects interact. 更难追踪哪些obj进行了交互 - Debugging can be more difficult. ✗ Overuse May Harm Design 损害设计 - Using a mediator for very simple interactions may not be needed. - Adds extra structure that might be unnecessary for small systems. 会增加小型系统不必要的额外结构
--	---

Strategy Pattern 策略模式 (Behavioral Pattern)

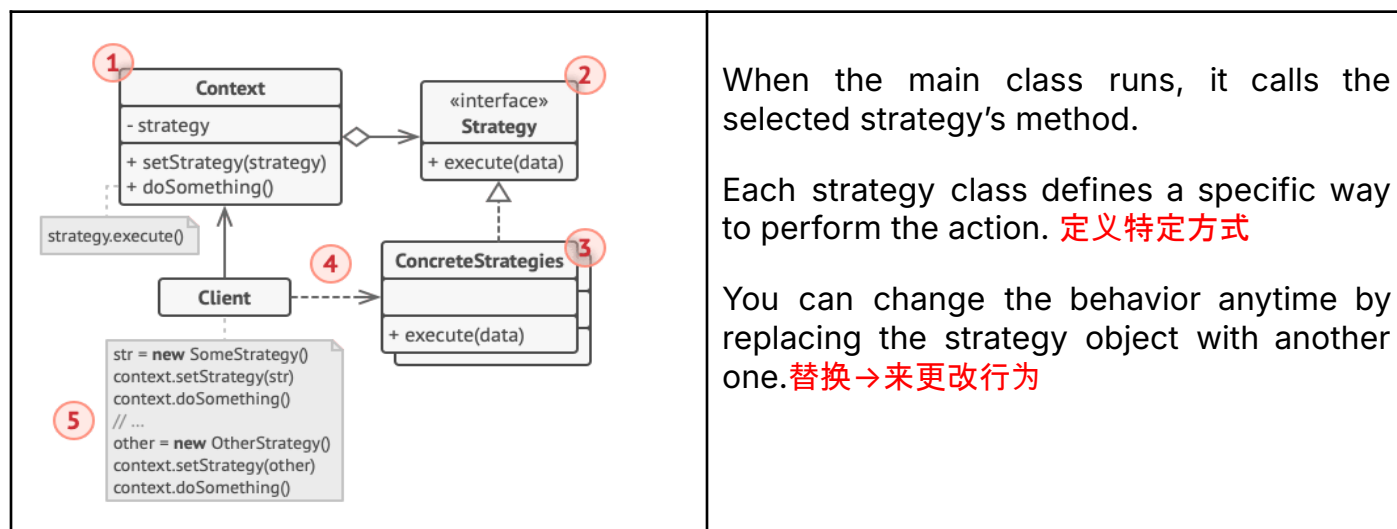
- Allows you to define different ways of doing something (strategies) and switch between them easily at runtime. 定义不同的执行方式(策略), 并在运行时轻松切换
- It lets you change the behavior 更改行为 of a class without changing its source code.
- It's like picking different routes for a journey based on conditions—same goal, different methods. 根据条件(相同的目标, 不同的方法)为旅程选择不同的路线

Problem Solved & Applicability

- Makes it easy to switch between multiple algorithms or behaviors without touching the core logic. 无需触及核心逻辑, 即可轻松在多种算法或行为之间切换
- Without this pattern, the class would be full of if-else or switch statements to handle each behavior. 将充斥着处理每种行为语句
- Strategy is applicable when you have different ways to complete a task and want to separate each variation into its own class. 有多种完成任务的方法, 并希望将每种方法拆分成各自的class

Solution (How It Works)

- **Create a class** that defines the overall goal or task and the steps needed to achieve it. 定义总体目标或任务以及实现该目标或任务所需的步骤
- Look at the steps and decide which parts vary depending on the chosen behavior—convert these steps into an abstract method. 查看这些步骤, 并确定哪些部分会根据所选行为而有所不同——将这些步骤转换为抽象方法
- Then, create **separate strategy classes** that implement those abstract methods to provide the different behaviors. 实现这些抽象方法, 以提供不同的行为
- The main class holds a **reference** to a strategy object and uses it to execute the task. 保存对策略obj的引用, 并使用它来执行任务
- This setup allows you to **easily swap strategies** at runtime without changing the logic of the main class. 允许您在运行时轻松切换策略, 而无需更改main class的逻辑



When the main class runs, it calls the selected strategy's method.

Each strategy class defines a specific way to perform the action. 定义特定方式

You can change the behavior anytime by replacing the strategy object with another one. 替换→来更改行为

How to Apply

- Identify the parts of the code that **change** depending on different situations (e.g., payment methods, sorting logic). 确定代码中根据不同情况(例如, 支付方式、排序逻辑)变化的部分
- Create a Strategy interface with a method to **represent the action**. 包含方法来表示操作
- Make **concrete classes**具体 for each strategy, each with a different implementation.
- In the main class, **store a strategy object** and call its method to complete the task. 存储一个策略对象并调用其方法来完成任务

When to Use

- You have a set of related behaviors, and each one can be chosen depending on user input or system condition. 有一组相关的行为, 并且可以根据用户输入或系统条件选择每个行为
- You want to avoid using long if-else blocks to decide how to perform a task. 避免使用冗长的 if-else 块来决定如何执行任务
- You want to make it easy to add new behaviors without touching the core logic. 能够轻松添加新行为, 而无需触及核心逻辑

Example: An e-commerce app allows users to select a payment method. 允许用户选择

- Each method (credit card, PayPal, etc.) is implemented as a separate strategy class. 单独的strategy class来的
- The app calls the selected strategy when the user confirms payment. 确认付款时, 应用会调用所选的策略

✓ Easy to Add New Strategies 轻松添加 - You can add a new behavior without changing existing code. 无需更改现有代码 - Each strategy is isolated and reusable. 独立且可重用 ✓ Easier Maintenance 更易于维护 - Replaces long if-else or switch logic. - Keeps the main class clean and focused. ✓ Improved Flexibility 提升灵活性 - You can change behavior while the program is running. 在程序运行时更改行为 - Useful when behavior depends on user input or settings. 行为依赖于用户输入或设置	✗ More Classes Needed - Each behavior needs its own class. - Increases the number of files and overall structure. 增加文件数量和整体结构 ✗ Can Hide Dependencies 隐藏依赖关系 - The main class must hold and manage the strategy object. 必须保存和管理 - Slight increase in setup complexity. 设置复杂性略有增加 ✗ Overuse May Harm Design 结构 - Using strategies for small differences may be overkill. 针对细微的差异使用策略可能会有些过度 - A simple method might work better for basic logic. 对于基本逻辑, 简单的方法可能更有效
---	--

Template Method Pattern 模板方法模式 (Behavioral Pattern)

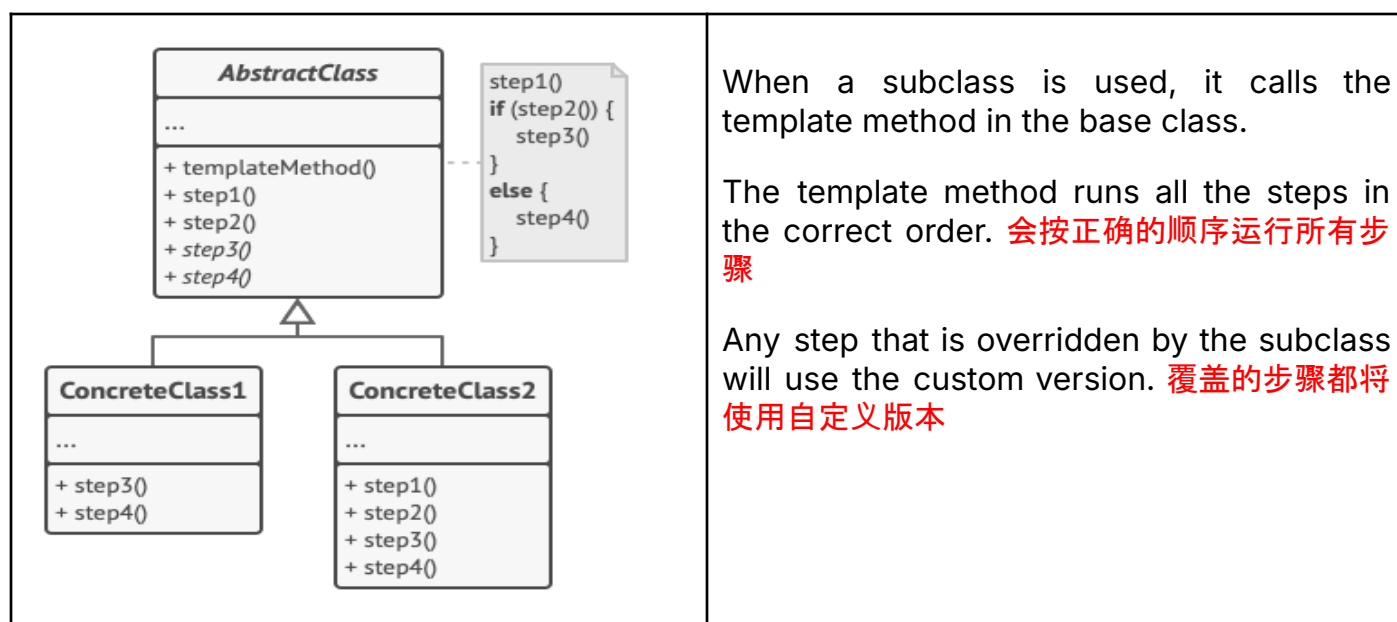
- Defines the **overall structure of an algorithm** 定义算法的整体结构 in a base class and lets subclasses change specific steps. 更改特定步骤
- It's useful when multiple classes share the same process but differ in only a few parts. 共享相同的流程但只有少数部分不同时
- It's like a recipe with fixed steps—only the ingredients or timing can change. 它就像一个步骤固定的菜谱——只有配料或时间可以更改

Problem Solved & Applicability

- Lets you keep common logic in one place while allowing subclasses to change only the steps that differ. 将通用逻辑集中在一处, 同时允许子类仅更改不同的步骤
- Without this pattern, shared logic would be copied in multiple classes, making updates harder. 共享逻辑将被复制 → 增加更新难度
- Template pattern is useful when you want **code reuse for a process with some variation** in certain steps. 希望对某些步骤存在差异的流程进行代码重用

Solution (How It Works)

- Create a base class that defines the full process as a method made up of several steps. 将整个流程定义为由多个步骤组成的方法
- Decide which steps always stay the same and which steps will vary—**turn the varying steps into abstract methods**. 确定哪些步骤始终保持不变, 哪些步骤会发生变化——将变化的步骤转换为抽象方法
- Then create subclasses that override the abstract methods to define their own behavior. 重写这些抽象方法以定义其自身的行为
- This way, the main structure stays the same, but each subclass can modify specific parts as needed. 主结构保持不变, 但每个subclass可以根据需要修改特定部分



How to Apply

- Find an algorithm or routine that is shared across multiple classes but has small differences. 找到一个在多个class之间共享但差异较小的算法或例程
- Create a base class with a template method that defines the steps of the process. 并使用模板方法定义流程的各个步骤
- Turn the different steps into abstract methods, and let each subclass provide its own version. 将不同的步骤转换为抽象方法, 并让每个subclass提供各自的版本
- This keeps the shared logic centralized and easy to manage. 共享逻辑集中化, 易于管理

When to Use

- Multiple classes follow the same structure, but some steps differ. flw相同的结构, 但某些步骤有所不同
- You want to **avoid repeating the same code** in each class and keep common logic in one place. 避免在每个class中重复相同的代码, 并将通用逻辑集中在一个地方
- You want to give subclasses **freedom to change specific steps** of a known process. 自由更改已知流程的特定步骤

Example: A cooking app has a general recipe format: boil water, add ingredients, and cook.

- Each dish has its own ingredient and cooking method. 自己的食材和烹饪方法
- The base class defines the steps, and each dish class overrides the ingredient and cooking parts. 每个菜谱类都会覆盖食材和烹饪步骤

✓ Code Reuse for Common Parts - Keeps shared logic in the base class. - Reduces duplication in subclasses. ✓ Easier Maintenance 易于维护 - Define steps in one place for easy understanding. 将步骤集中定义, 方便理解 - Each subclass only focuses on changing specific behavior. 仅关注特定行为的更改 ✓ Improved Flexibility - Add new behavior by creating a new subclass. 创建新的 → 添加新行为 - No need to change the base class. 方便理解	✗ Too Many Subclasses - Each variation needs a separate subclass. - Can lead to class explosion if overused. 方便理解 ✗ Harder to Track Behavior 行为追踪难度加大 - Logic is split between base class and subclasses. 逻辑被拆分 - Reading the full process may require checking both classes. 读取完整流程需要检查2边 ✗ Overhead for Small Processes 小型流程的开销 - Not suitable for simple algorithms. 不适用于简单的算法 - A single method might be better in basic cases. 在基本情况下, 单一方法可能更好
---	--

Strategy vs Template Method: What's the Difference?

Main Difference

- Strategy uses **composition** 组合: pass in the strategy object to the main class.
- Template Method uses **inheritance** 继承: extend the base class and override steps. 扩展基类并覆盖步骤

Who Controls the Steps?

- In Strategy, the **main class** decides which strategy to use at runtime. 决定运行时用哪种策略
- In Template, the **base class** decides the overall flow, but allows subclasses to customize parts. 决定整体流程, 但允许子类自定义部分内容

How to Choose

Use **Strategy Pattern** when:

- want to **switch behaviors at runtime** 希望在运行时切换行为
- need to **change the algorithm without touching the main class** 需要在不修改主类的情况下更改算法
- prefer **composition** over inheritance 更喜欢组合而不是继承

Use **Template Pattern** when:

- want to **share the structure of a process** across classes 希望跨类共享流程结构
- have multiple classes doing almost the same thing 有多个类执行几乎相同的操作
- prefer **inheritance** and central control of the process 更喜欢继承和对流程的集中控制

Facade Pattern 外观模式 (Structural Pattern)

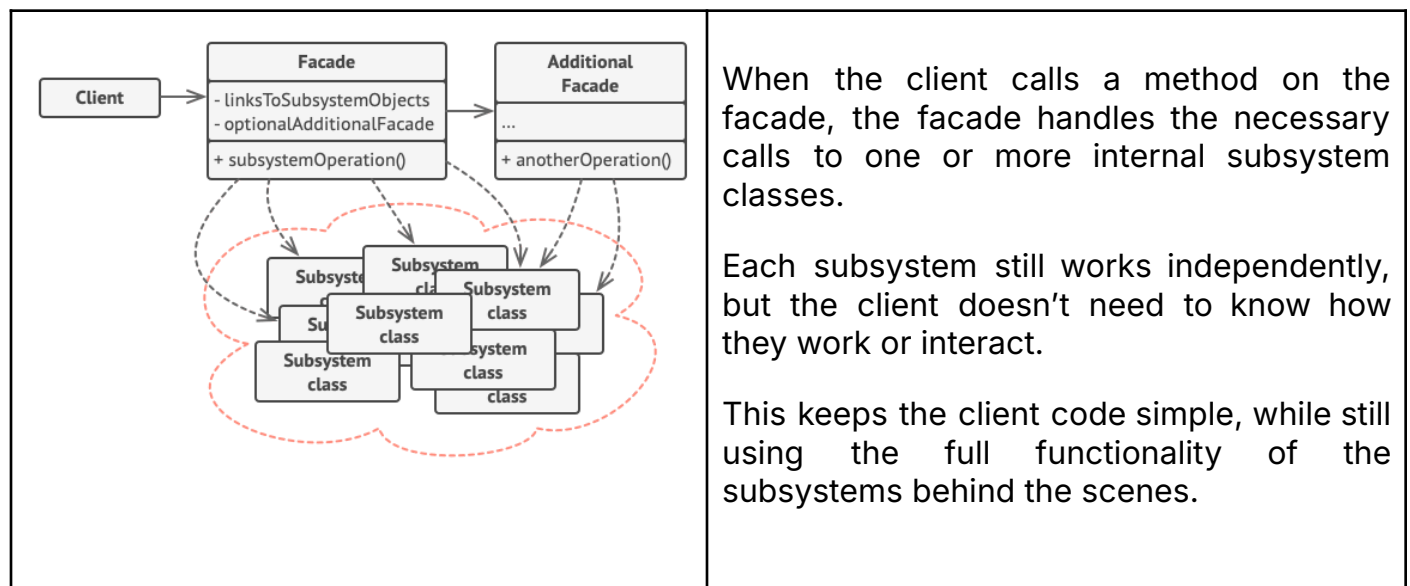
- Provides a simplified interface to a group of related classes that are complex or have many interactions. 为一组复杂或交互繁琐的相关class提供简化的接口
- It helps **reduce the difficulty of using** a system by offering a single, easy-to-use entry point. 降低系统的使用难度 ~ 提供单一、易于使用的入口点
- It's like using a remote control to manage multiple internal components with just one button. 这就像使用遥控器, 只需一个按钮即可管理多个内部组件

Problem Solved & Applicability

- Hides the details of multiple subsystems behind one unified interface. 隐藏在一个统一的接口后
- Using several classes directly can be overwhelming, especially when they have complex interactions. 直接使用多个class可能会让人不知所措, 尤其是在它们具有复杂的交互时
- Facade is useful when you want to provide a **clean and simple interface** to a complicated group of classes or a legacy system 想要为一组复杂的类或遗留系统提供简洁明了的接口

Solution (How It Works)

- Create a new class called the **Facade**, which has methods that internally call several methods from the subsystem classes. 包含一些方法, 可以在内部调用子系统类中的多个方法
- The client only interacts with the facade, not the underlying subsystems. 客户端只与 Facade 交互, 而不与底层子系统交互
- This helps reduce the number of connections in the code and makes the system easier to use and maintain. 有助于减少代码中的连接数量, 并使系统更易于使用和维护



How to Apply

- Identify a set of classes that often work together and are frequently used in combination. 确定一组经常协同工作且经常组合使用的class
- Design a facade class with methods that represent common high-level operations the client needs. 设计一个FC, 其方法代表客户端所需的常见高级操作
- In each facade method, call the necessary subsystem methods in the **correct order**. 按正确的顺序调用必要的子系统方法
- This gives users a **clean interface** and hides the complexity from them. 提供了一个简洁的界面, 并隐藏了复杂性

When to Use

- You are working with a **complex system** made up of many classes and want to provide a **simple interface** for client code. 希望为客户端代码提供一个简单的接口
- You want to **isolate** 隔离 the client from system internals, such as in **libraries, APIs, or legacy codebases**. 客户端与系统内部
- You want to **make the system easier to use or replace**, without changing client code. 不更改客户端代码的情况下, 使系统更易于使用或替换

Example: Imagine a home theater system 家庭影院系统 with separate classes for the DVD player, projector, speakers, and lights.

- Instead of calling each class separately to turn on the system, a **HomeTheaterFacade** class can perform all actions with one method. 无需分别调用每个类来启动系统, 而是可以通过一个方法执行所有操作
- The client just calls `startMovie()` on the facade, and it sets up everything in the right order. 它就会按照正确的顺序设置所有内容

✓ **Simplifies Usage 简化使用**

- Reduces the number of interactions needed to use the system. 减少使用系统所需的交互次数
- Makes complex code easier to understand and maintain. 使复杂代码更易于理解和维护

✓ **Hides Internal Complexity 隐藏内部复杂性**

- Keeps the client code clean and free from subsystem details. 保持客户端代码简洁, 不受子系统细节影响
- Helps isolate the client from system changes. 有助于将客户端与系统变更隔离

✓ **Improves Code Organization 改进代码组织**

- Provides a clear separation between client-facing operations and internal workings. 明确区分面向客户端的操作和内部工作
- Makes high-level tasks easier to manage and reuse. 使高级任务更易于管理和重用

✗ **May Limit Access to Features 可能限制功能访问**

- Some advanced functions may not be exposed by the facade. 某些高级功能可能无法通过 Facade 公开
- Clients might need to access subsystems directly if the facade is too simplified. 过于简化, 客户端可能需要直接访问子系统

✗ **Risk of Too Much Responsibility**

- If the facade tries to do too much, it can become bloated. 试图执行过多操作, 可能会变得臃肿
- Hard to maintain if it grows into a "God class." 发展成为"上帝类", 则难以维护

✗ **Overhead for Small Processes 小型进程的开销**

- Facade should not contain business logic itself. 本身不应包含业务逻辑
- It is only meant to **coordinate calls**, not **process data**. 仅用于协调调用, 而不是处理数据